Mohammed Mufthi

IT22355478

Undergrad Thesis Checking

Document Details

Submission ID

trn:oid:::3618:136605646

Submission Date

Apr 26, 2026, 11:55 AM GMT+5:30

Download Date

Apr 26, 2026, 11:57 AM GMT+5:30

File Name

IT22355478.pdf

File Size

806.7 KB

54 Pages

9,546 Words

60,455 Characters

6% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography
- Quoted Text
- Small Matches (less than 8 words)

Match Groups

38 Not Cited or Quoted 6%
Matches with neither in-text citation nor quotation marks

1 Missing Quotations 0%
Matches that are still very similar to source material

0 Missing Citation 0%
Matches that have quotation marks, but no in-text citation

0 Cited and Quoted 0%
Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
1% Publications
5% Submitted works (Student Papers)

Match Groups

- **38** Not Cited or Quoted 6%
  Matches with neither in-text citation nor quotation marks
- **1** Missing Quotations 0%
  Matches that are still very similar to source material
- **0** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

5% Internet sources
1% Publications
5% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Student papers	Sri Lanka Institute of Information Technology on 2021-10-09	2%
2	Student papers	National Institute of Business Management Sri Lanka on 2023-08-25	<1%
3	Internet	www.grin.com	<1%
4	Student papers	University of Bedfordshire on 2026-04-23	<1%
5	Internet	transweb.sjsu.edu	<1%
6	Student papers	Middle East Technical University on 2012-05-14	<1%
7	Internet	research.library.mun.ca	<1%
8	Student papers	Sri Lanka Institute of Information Technology on 2017-12-02	<1%
9	Publication	K. V. Sambasivarao, Anasuya Sesha Roopa Devi Bhima. "Artificial Intelligence, Co...	<1%
10	Internet	github.com	<1%

11 Student papers

Liverpool John Moores University on 2026-04-08 <1%

12 Student papers

University of Wales Institute, Cardiff on 2012-01-05 <1%

13 Internet

archive.org <1%

14 Internet

manuals.muthu.co <1%

15 Internet

www.moe.go.tz <1%

16 Student papers

Sri Lanka Institute of Information Technology on 2021-10-08 <1%

17 Student papers

University of Wales Institute, Cardiff on 2024-11-22 <1%

18 Publication

Adam Julius, James S. Bowness. "Generative AI models: the next anaesthetic agen... <1%

19 Student papers

Lahore Garrison University-Lahore. on 2026-04-13 <1%

20 Internet

dr.ur.ac.rw <1%

21 Student papers

Brunel University on 2025-04-11 <1%

22 Student papers

Faculty of Engineering, Chulalongkorn University on 2025-12-06 <1%

23 Student papers

University of Greenwich on 2025-11-29 <1%

24 Internet

fastercapital.com <1%

25 Student papers

Sri Lanka Institute of Information Technology on 2021-10-13 <1%

26 Internet

ijircce.com <1%

27 Internet

m.moam.info <1%

28 Student papers

Asian Institute of Technology on 2010-11-08 <1%

29 Student papers

University of Birmingham on 2011-09-02 <1%

30 Student papers

Islington College,Nepal on 2026-04-22 <1%

31 Student papers

Kingston University on 2025-09-18 <1%

32 Student papers

University of Westminster on 2025-02-02 <1%

33 Internet

dl.lib.uom.lk <1%

34 Internet

dspace.cuni.cz <1%

35 Internet

repository.kisiiuniversity.ac.ke:8080 <1%

LEVERAGING AGENTIC AI MIDDLEWARE FOR INTELLIGENT INTERACTION AND MODERNIZATION OF LEGACY ENTERPRISE APPLICATIONS

M.A.M.Mufthi

(IT22355478)

2

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

May 2026

LEVERAGING AGENTIC AI MIDDLEWARE FOR INTELLIGENT INTERACTION AND MODERNIZATION OF LEGACY ENTERPRISE APPLICATIONS

M.A.M.Mufthi

(IT22355478)

1

Dissertation submitted in partial fulfillment of the requirements for the Bachelor of Science in Information Technology Specializing in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology
Sri Lanka

May 2026

DECLARATION

We declare that this is our own work, and this dissertation does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or Institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, we hereby grant to Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute our dissertation, in whole or in part in print, electronic or other medium. We retain the right to use this content in whole or part in future works (such as articles or books).

Name	Student ID	Signature
M A M Mufthi	IT22355478	

25

Signature of the Supervisor: ……

(Mr. Vishan Jayasinghearachchi)

Signature of the Co-Supervisor: ……

(Dr. Dharshana Kasthurirathna)

Date: 25/04/2026

ABSTRACT

Legacy information systems underpin critical operations in sectors like finance and healthcare but are hindered by archaic, GUI-only interfaces that prevent automation and integration with modern platforms. Organizations face severe operational inefficiencies, with a significant portion of their digital transformation budget consumed by maintenance rather than innovation. Traditional modernization methods are prohibitively expensive and risky, while conventional Robotic Process Automation (RPA) tools are too brittle and lack the intelligence to adapt to dynamic environments or provide user-facing transparency.

This study proposes a novel, agentic AI middleware framework the Legacy Application Modernization Architecture (LAMA) to intelligently automate and modernize legacy applications without modifying their source code. The core of this system is the GUI Interaction Task Translator (GITT), a component that captures user inputs from both manual forms and uploaded document images and translates them into high-level intents. These intents are decomposed into executable tasks, orchestrated across adaptive RPA agents, and executed on the legacy system. The framework incorporates a self-correcting feedback loop and real-time monitoring to ensure resilience and build user trust. This approach provides a practical, non-intrusive, and scalable path to legacy modernization, enhancing operational efficiency and enabling seamless integration into the modern digital ecosystem.

Keywords: Agentic AI, Legacy System Modernization, Robotic Process Automation (RPA), GUI Automation, Middleware, Task Orchestration, GITT, LAMA

35

DEDICATION

This dissertation is dedicated to my family, supervisors, and all those whose unwavering support and encouragement made this work possible.

ACKNOWLEDGEMENTS

First and foremost, I would like to express my sincere gratitude to my supervisors, Mr. Vishan Jayasinghearachchi and Dr. Dharshana Kasthurirathna, of the Department of Computer Science and Software Engineering at the Sri Lanka Institute of Information Technology. Their unwavering guidance, timely feedback, and technical expertise were indispensable at every stage of this research. Their encouragement during the most challenging phases of the project particularly during the integration of the multi-agent orchestration layer with the legacy VM environment gave me the confidence and direction needed to persevere.

I also extend my heartfelt appreciation to the co-supervisors and panel members who provided constructive critique during the proposal and progress reviews. Their suggestions shaped the experimental evaluation methodology and strengthened the scientific rigour of the research findings. Special thanks go to the industry professionals from the healthcare IT sector in Sri Lanka who shared their operational experiences with legacy hospital management systems, providing invaluable practical context that grounded the research problem in real-world constraints.

I am deeply grateful to the technical staff of the SLIIT Computing Infrastructure team for their assistance in configuring the virtualized evaluation environment, including the Oracle VirtualBox instance running Windows XP SP3 used for evaluation. Access to this infrastructure was essential for the empirical component of this research.

Finally, I thank my family and friends, whose moral support throughout the long hours of development, testing, and writing sustained me through this journey.

10

TABLE OF CONTENTS

28

DECLARATION iii
ABSTRACT iv
DEDICATION v
ACKNOWLEDGEMENTS vi
LIST OF FIGURES x
LIST OF ABBREVIATIONS xii
1. INTRODUCTION 1
13
1.1 Background and Literature Survey 2
1.1.1 Background 2
1.1.2 Literature Survey 3
1.2 Research Gap 5
3
1.3 Research Problem 6
1.4 Objectives 8
1.4.1 Main Objective 8
1.4.2 Specific Objectives 8
1.5 Scope and Boundaries 8
2. METHODOLOGY 9
2.1 Overall System Architecture 9
2.2 GUI Interaction Task Translator (GITT) Component 11
2.2.1 Input Capture and Task Recognition 12
2.2.2 Intent Translation to Legacy Commands 13
2.2.3 Feedback and Self-Correction Loop 14
2.2.4 Real-Time Monitoring and Reporting 14
2.3 Research Process 15
21
2.3.1 Software Development Methodology 15
2.3.2 Requirements Gathering 15
2.3.3 Survey and Observation 16
2.3.4 Planning and Architecture 16
2.4 AI Output Reliability Strategy 16
2.5 LLM Integration, Rationale, and Migration from Helper Functions 17
2.5.1 Why an LLM was Adopted 18
2.5.2 Why the Project Switched from Helper-Only Extraction 18

2.5.3 How the LLM Detects UI Structure 19
2.6 Application Overview and User Workflow 20
2.6.1 System Overview 21
2.6.2 Application Phases and Screenshots 21
2.7 Implementation 25
6 2.8 Software Requirements 26
2.8.1 Functional Requirements 26
2.8.2 Non-Functional Requirements 26
2.8.3 System Requirements 27
2.9 Commercialization Strategy 27
Phase 1: Beta Deployment 28
Phase 2: Limited Commercial Release 28
Phase 3: Enterprise Scale Deployment 28
2.10 Testing and Implementation 29
2.10.1 Testing Strategy 29
2.10.2 System Testing – End-to-End Clinical Workflows 29
2.10.3 Evaluation Metrics 29
2.10.4 Gantt Chart 30
8 3. RESULTS AND DISCUSSION 31
3.1 Overall Evaluation Results 31
3.2 Feature Comparison with Related Approaches 31
3.3 Summary of System Architectures and Technologies 32
3.4 Research Findings 33
Finding 1: Runtime UI Capture is a Viable Modernization Signal 33
Finding 2: Multimodal AI Can Accelerate Structural Extraction but Requires Guardrails 33
Finding 3: Iterative Capture Improves Completeness 33
Finding 4: Human Validation Remains Essential 33
Finding 5: Agentic Self-Correction Reduces Manual Intervention 34
3.5 Discussion 34
12 3.5.1 Strengths 34
3.5.2 Limitations 34
3.5.3 Threats to Validity 35
3.5.4 Summary of Student Contribution 35
4. CONCLUSION 36

REFERENCES 38
APPENDICES 40
Appendix A - System Workflow Summary 40
Appendix B – Glossary 41

5

LIST OF FIGURES

Figure 1 – High- Level System Architecture Diagram 11
Figure 2 - System Execution Flowchart 11
Figure 3 - GUI Interaction Task Translator System Diagram 12
Figure 4 - Builder app: Getting Started 22
Figure 5 - Builder App: Screenshot Capture Trigger and Scan Progress 22
Figure 6 - Builder App: Schema Preview — Detected Pages, Columns, and Fields 23
Figure 7 - Builder App: Frontend Generation Configuration Screen 24
Figure 8 - Generated React Frontend - Modern Web Interface from Legacy Schema 24
Figure 9 - Gantt Chart: Development Timeline (Jul 2025 – Jun 2026)] 30

33

LIST OF TABLES

Table 1 - Research Gap Analysis 6
Table 2 - AI Output Reliability: Defensive Post-Processing Steps 17
Table 3 - LLM-Based vs. Helper-Function Extraction: Comparative Analysis 20
Table 4 - Software and System Requirements 27
Table 5 - Project Budget Estimate 28
Table 6 - Feature Comparison with Related Approaches 32
Table 7 - System Architecture and Technologies Summary 32

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
API	Application Programming Interface
CNN	Convolutional Neural Network
CRUD	Create, Read, Update, Delete
GITT	GUI Interaction Task Translator
GUI	Graphical User Interface
JSON	JavaScript Object Notation
LAMA	Legacy Application Modernization Architecture
LLM	Large Language Model
NLP	Natural Language Processing
OCR	Optical Character Recognition
RPA	Robotic Process Automation
SCRUM	Agile Software Development Framework
SLIIT	Sri Lanka Institute of Information Technology
SME	Small and Medium Enterprise
SOACRS	Self-Optimizing Agent Coordination and Response System
UI	User Interface
VM	Virtual Machine
WebSocket	Web Communication Protocol (RFC 6455)

LIST OF APPENDICES

Appendix A - System Workflow Summary 40
Appendix B - Glossary 41

1. INTRODUCTION

Legacy information systems continue to underpin mission-critical operations in sectors such as healthcare, finance, transportation, and public administration, despite being built on architectures and technologies that are often several decades old. These systems are stable and deeply embedded in organizational workflows, supporting essential business operations that range from patient record management and banking transactions to government service delivery. However, their stability comes at the cost of flexibility, scalability, and integration capabilities.

Many legacy applications rely on proprietary, GUI-only interfaces, preventing straightforward integration with modern cloud platforms, mobile applications, or real-time analytics solutions. The inability to adapt or interoperate with contemporary technologies creates substantial barriers for organizations attempting to digitize their operations. Additionally, legacy systems often lack comprehensive documentation, modularity, or standardized interfaces, making even minor updates or expansions complex, error-prone, and expensive.

This dissertation presents the Legacy Application Modernization Architecture (LAMA), a modular, agentic AI middleware framework designed to intelligently automate and modernize legacy desktop applications without modifying their source code. The core component the GUI Interaction Task Translator (GITT) captures user inputs from modern interfaces and uploaded document images, translates them into high-level intents, and orchestrates their execution on legacy systems through adaptive RPA agents. The framework incorporates self-correcting feedback loops and real-time monitoring to ensure resilience and user trust.

27

1.1 Background and Literature Survey

1.1.1 Background

Legacy information systems remain critical in sectors such as healthcare, finance, transportation, and public administration. Despite their stability, these systems were designed decades ago with monolithic architectures, rigid workflows, and proprietary GUI-only interfaces, making them difficult to modernize or integrate with contemporary digital platforms. Organizations are often forced to maintain these systems at high cost, dedicating nearly 70% of digital transformation budgets to upkeep rather than innovation. Technical debt, coupled with the absence of modularity and limited interoperability, restricts agility and slows the adoption of emerging technologies.

Traditional modernization strategies including re-hosting, re-platforming, and complete system re-engineering attempt to improve integration and operational performance but come with significant costs, extended timelines, and high technical risks. Re-hosting improves infrastructure efficiency but does little to enhance application logic or interface flexibility. Re-platforming often requires significant modification of codebases that may be undocumented or poorly understood, introducing operational failure risks. Full system re-engineering can produce highly flexible, modernized applications but is prohibitively expensive, often taking months or years.

Robotic Process Automation (RPA) tools, including UiPath and Automation Anywhere, have emerged as an attractive non-intrusive approach for automating interactions with GUI-only systems. RPA simulates human input by mimicking mouse clicks, keystrokes, and form interactions, enabling organizations to automate repetitive tasks without modifying underlying legacy code. However, conventional RPA implementations are highly dependent on static GUI element selectors or absolute screen coordinates, which are prone to failure if the interface layout changes even slightly. These solutions require extensive manual configuration, are difficult to scale,

and provide limited adaptability in dynamic environments. Furthermore, traditional RPA lacks intelligence to interpret high-level operational goals or recover from unexpected failures.

1.1.2 Literature Survey

Several research efforts have explored legacy system modernization, automation, and agentic AI. Kazman et al. [1] focused on architecture reconstruction and re-engineering, demonstrating techniques to analyze legacy systems and extract functional structures. Their approach enabled the identification of hidden dependencies, modular boundaries, and potential bottlenecks, laying groundwork for non-intrusive modernization approaches.

Gartner [2] highlighted the operational and financial challenges in legacy modernization, emphasizing that maintaining old systems consumes most digital transformation budgets while leaving little room for innovation. The report underscores the practical need for middleware frameworks that automate tasks and interface with legacy systems efficiently, urging adaptable solutions that reduce downtime while enabling integration with modern platforms.

29

Khajeh-Hosseini et al. [3] proposed the cloud adoption toolkit, offering methodologies to evaluate, migrate, or integrate legacy applications into cloud environments. While effective for structured decision-making in re-platforming, it still requires significant manual effort, highlighting the gap for autonomous, adaptive middleware solutions that can dynamically interpret workflows without deep source code knowledge.

Lacity and Willcocks [4] analyzed RPA solutions and their capabilities in automating GUI-based workflows. Their study revealed that while RPA can reduce manual effort, conventional approaches struggle with interface changes, lack self-healing, and are prone to errors — emphasizing the necessity of integrating intelligent task orchestration and feedback mechanisms into RPA pipelines.

Dustdar et al. [5] discussed software-defined cloud manufacturing, illustrating modular, agent-based middleware architectures for complex system orchestration. Their findings demonstrate the benefits of decoupled, modular components that can dynamically coordinate multiple tasks and agents a concept directly applicable to agentic middleware for legacy GUI modernization.

Wang et al. [6] proposed intelligent task planning for human-computer interaction systems, showing how AI can decompose high-level goals into smaller, structured subtasks. Their approach allows systems to understand workflow dependencies and optimize task execution, forming a basis for the planning and translation strategies employed in GITT.

Oberhauser and Pogolski [7] presented adaptive RPA systems for GUI-only legacy applications, emphasizing resilient automation through AI-driven macros. While these systems could handle interface changes, they lacked support for high-level CRUD operations, manual form entry, image-based inputs, or real-time user feedback. GITT addresses these shortcomings by providing a central interface for users with deterministic mapping of interactions, integration with adaptive RPA agents, and continuous monitoring with feedback loops.

Gu et al. [8] introduced middleware-based automation for legacy GUI systems using AI orchestration, presenting a complete framework integrating task planning, GUI element recognition, and adaptive RPA. Their methodology validates the concept of a central translation layer such as GITT that bridges high-level task planning and GUI execution.

Additional studies include: Chen et al. [9], who applied CNN-based screen parsing for GUI element identification; Zhang et al. [10], who developed task decomposition pipelines for automated desktop applications; Patel et al. [11], who demonstrated

feedback-driven execution refinement; and Singh et al. [12], who explored modular RPA orchestration for multi-application legacy environments. Collectively, these works form the technical foundation for GITT.

1.2 Research Gap

Despite significant advances in automation, legacy system modernization, and agentic orchestration, existing research exhibits several critical limitations when addressing GUI-driven legacy systems in a fully integrated, user-facing manner.

Smith et al. [13] explored NLP-based query translation frameworks for converting natural language inputs into structured system commands. While effective for database interactions, their scope is confined to structured database contexts rather than GUI-based legacy systems. They offer no mechanism for translating natural language queries into interactive GUI workflows involving navigation, element recognition, and task execution.

Li et al. [14] investigate middleware architectures for legacy system modernization, highlighting the role of modular integration layers. However, their middleware serves primarily as a communication layer; end-user tasks involving graphical interfaces remain outside its scope. The GITT framework advances this by embedding GUI-driven task translation within the middleware itself.

Wang et al. [15] present task automation frameworks that focus on workflow execution through rule-based engines. Their approach lacks adaptability to dynamic GUIs and does not integrate real-time intent recognition or self-correction mechanisms. GITT addresses this by incorporating adaptive orchestration that dynamically adjusts workflows when GUI layouts change.

Oberhauser and Pogolski [7] propose adaptive RPA systems capable of self-healing automation. This significantly improves robustness over traditional RPA but remains constrained to backend execution pipelines without meaningful user-facing interaction, high-level task translation, or multimodal input support.

Taken together, these gaps highlight the need for a system that combines deterministic intent recognition, adaptive GUI task translation, multimodal input handling, and continuous feedback in a single self-optimizing middleware component. The following table summarizes the research gaps addressed by the proposed system.

Table 1 - Research Gap Analysis

Research / System	Research [13]	Research [14]	Research [15]	Research [7]	Proposed System
Dual Input (Manual+Image)	✗	✗	✗	✗	✓
Task Breakdown	✓	✓	✓	✓	✓
Real-Time Tracking	✗	✗	✗	✗	✓
Integration with Legacy Systems	✓	✓	✗	✓	✓

1.3 Research Problem

Modern enterprises continue to rely on complex software ecosystems — including legacy GUI-only applications, modern digital platforms, and enterprise workflow tools — to manage critical business operations. Many legacy systems remain indispensable despite being rigid, lacking integration capabilities, and offering no programmatic interfaces. These systems introduce persistent challenges:

- Manual Operation and Limited Automation: Legacy GUI systems require direct user interaction for routine tasks such as adding, updating, or retrieving records. This manual dependency leads to inefficiencies, errors, and

operational bottlenecks, especially when workflows are repetitive or data intensive.

- Integration Complexity: Legacy systems typically lack APIs or code hooks, making it difficult to connect them to modern orchestration or analytics tools. Coordinating actions across multiple systems requires careful simulation of GUI interactions that traditional automation frameworks cannot fully support.
- Limited Adaptability and Intelligence: Conventional RPA solutions depend on static macros and hard-coded selectors. They fail when interface elements shift or workflows evolve, and they provide no mechanism for dynamically interpreting high-level user inputs or scanning images of forms to extract structured data.
- Scalability and Performance Constraints: Traditional automation frameworks struggle to scale for complex workflows, multiple concurrent tasks, or dynamically generated operations. Maintaining reliability and low-latency execution across many legacy applications is challenging without intelligent task orchestration.
- Data Accuracy, Reliability, and Monitoring: Ensuring correct execution of user tasks and providing transparency is difficult in legacy systems. Without real-time feedback, error tracking, and adaptive self-correction, automation often fails silently or requires continuous human intervention.

The central research problem addressed is: How can an intelligent, user-facing, and adaptive middleware system translate structured user inputs including manual entries and scanned images into actionable GUI tasks on legacy systems, while providing real-time monitoring, adaptive self-correction, and minimal human intervention?

15

1.4 Objectives

1.4.1 Main Objective

The main objective is to design and implement an intelligent, modular, and non-intrusive modernization framework for legacy GUI-only systems by leveraging agentic artificial intelligence, low-code Robotic Process Automation (RPA), and autonomous coordination mechanisms. The framework will incorporate a GUI interaction task translation module capable of decomposing structured task inputs into granular GUI actions, mapping them to executable RPA commands, and enabling autonomous execution of complex workflows — ensuring seamless integration with modern platforms without modifying the original system source code.

1.4.2 Specific Objectives

1. Implement deterministic intent recognition from user interactions by developing the GUI Interaction Task Translator (GITT) to capture structured front-end actions including manual inputs and image uploads for form recognition.
2. Translate recognized intents into executable legacy system commands by converting identified intents into specific sequences of legacy system operations without modifying the underlying system.
3. Integrate a self-correction mechanism that continuously validates the outcomes of executed commands and automatically re-maps and adjusts commands when errors occur.
4. Establish a feedback and refinement loop for continuous learning by collecting execution results, error logs, and user confirmations to improve future mappings between GUI actions, intents, and legacy commands.
5. Enable real-time execution, monitoring, and task tracking by providing low-latency translation and execution with continuous monitoring visible to users.

1.5 Scope and Boundaries

This research focuses on non-intrusive legacy modernization through agentic AI middleware. The current prototype addresses:

- Detection and automation of visible UI structures in legacy GUI applications (forms, tables, input fields, navigation menus).
- Multi-modal input processing including structured form entries and scanned document images processed through OCR.
- End-to-end workflow automation from user intent capture to legacy system interaction and result reporting.
- Frontend scaffold generation from captured legacy screenshots to provide a modern web interface for users.

Out of scope for this version include automatic extraction of deep business rules from backend binaries, guaranteed one-click production deployment, full security and compliance hardening for enterprise release, and support for non-GUI legacy systems using mainframe or batch-processing interfaces.

2. METHODOLOGY

This research follows a design-and-build methodology with iterative refinement, guided by the Agile SCRUM framework. The methodology encompasses the overall system architecture design, detailed implementation of core components, the research process for requirements gathering and validation, and the commercialization strategy for the resulting product.

2.1 Overall System Architecture

The Legacy Application Modernization Architecture (LAMA) modernizes legacy desktop applications without modifying their source code. The framework establishes a complete, layered bridge between the legacy application and modern users, comprising four principal components that operate in sequence: the Frontend Builder (incorporating GITT), the Self-Optimizing Agent Coordination and Response System (SOACRS), the Orchestrator Agent, and the RPA Bot Layer. Each component has a clearly defined interface with its neighbors, enabling independent development, testing, and replacement without disrupting the overall system.

The overall system operation proceeds as follows. An administrator captures a screenshot of each target legacy application screen using PyAutoGUI. The Frontend Builder submits each screenshot to a multimodal LLM with a structured prompt requesting JSON extraction of the screen's UI components. The resulting JSON schema feeds a code-generation pipeline that produces a React web application, deployed on a Node.js server accessible from any device. When a user interacts with the generated interface, each action generates a structured JSON event transmitted via Secure WebSocket to SOACRS.

SOACRS receives the event, identifies the task type and context, evaluates all registered automation tools against its composite scoring function, and generates an ordered state-machine execution plan. The plan is transmitted to the Orchestrator, which captures a screenshot of the legacy application, verifies pre-conditions for the first step, and dispatches the appropriate RPA bot. The bot executes its assigned interaction using the three-strategy hybrid input injection chain. After completion, the Orchestrator captures another screenshot and verifies post-conditions. If verification passes, the next step is dispatched; if not, the retry or fallback mechanism is invoked. The final result is returned to the frontend via WebSocket as a structured response.

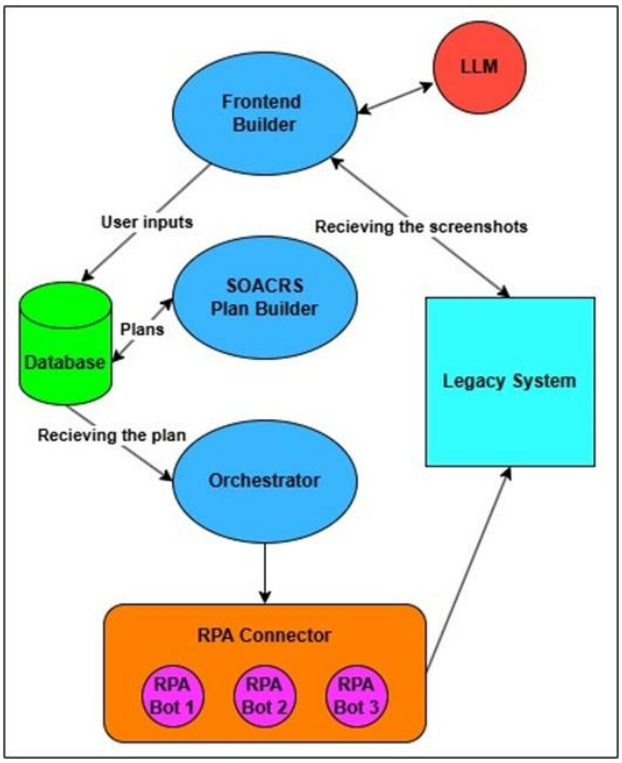

Figure 1 – High- Level System Architecture Diagram

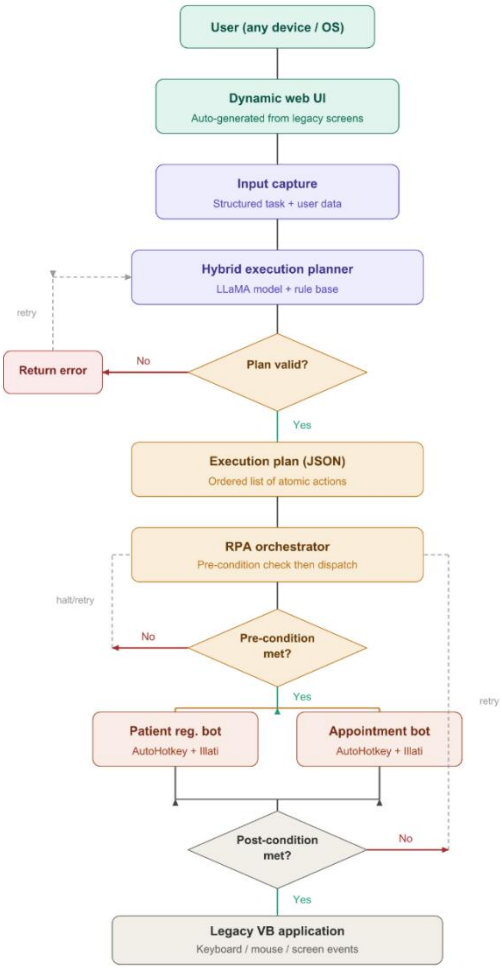

Figure 2 - System Execution Flowchart

2.2 GUI Interaction Task Translator (GITT) Component

The GUI Interaction Task Translator (GITT) serves as the central user-facing component of the proposed system, directly bridging user interactions with legacy system execution. Unlike conventional middleware or natural language-based

automation, GITT acts as an interface layer where users can perform tasks through structured manual input (via buttons, forms, and actions) or by uploading scanned images/forms for automatic recognition.

Once inputs are captured, GITT translates them into structured prompts and commands that downstream orchestrator agents can process, which are then executed in the legacy systems through adaptive RPA modules. Importantly, GITT not only translates tasks but also provides real-time monitoring, feedback, and self-correction mechanisms, ensuring reliability, adaptability, and transparency for the user.

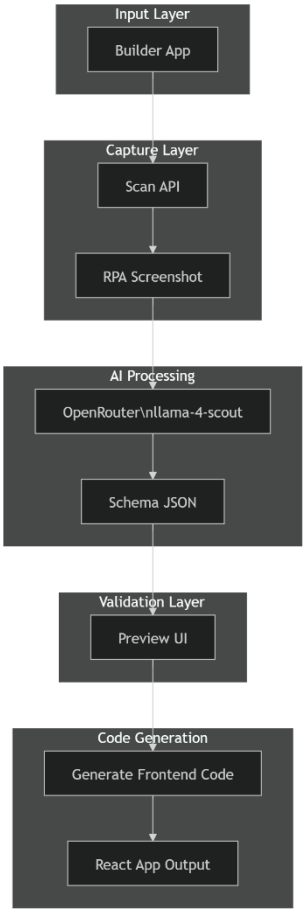

Figure 3 - GUI Interaction Task Translator System Diagram

2.2.1 Input Capture and Task Recognition

In the proposed framework, accurate interpretation of user interactions is crucial for reliable automation of legacy systems. GITT is responsible for understanding both

direct user actions and visual inputs, converting them into high-level intents executable by downstream modules. GITT receives input through two main channels:

1. Structured Event Capture: The interface provides intuitive options such as Add, Delete, Update, and Retrieve buttons, allowing users to initiate specific actions. Once an action is selected, the system dynamically generates the corresponding input fields required for data entry. The information entered is captured and encoded into structured event objects enriched with metadata including action type, field names, and input values, ensuring accurate and organized processing.
2. Image-Based Form Input: Users can upload scanned forms or screenshots of legacy system documents, which are processed through enhancement techniques such as contrast adjustment, skew correction, and noise removal to improve readability. Optical Character Recognition (OCR) is then applied to extract text and form data, converting them into structured input fields integrated into the task translation workflow.

The captured input is sent to the Intent Recognition Module, which maps each action or identified form element to a high-level system intent using both deterministic rules for standard elements and AI-based inference for complex or ambiguous inputs.

2.2.2 Intent Translation to Legacy Commands

After GUI elements and user intents have been accurately recognized, the system translates these intents into actionable commands that the legacy system can execute. The recognized intents are processed by the Task Translation Module, which converts them into executable commands compatible with the legacy system through the following key steps:

- Mapping high-level intents to low-level GUI operations.

- Determining action sequences, including navigation steps, field population, button clicks, and menu interactions.
- Resolving any ambiguities using historical interaction logs and context from previous executions.

This translation layer is designed to abstract away system-specific complexities, ensuring compatibility across multiple legacy applications without requiring source code modifications.

2.2.3 Feedback and Self-Correction Loop

The Feedback and Self-Correction Loop serve as the adaptive core of GITT, ensuring that task translations remain accurate and reliable even in dynamic or evolving legacy environments. By continuously observing the outcomes of executed actions, the system detects deviations from expected behavior, identifies root causes of errors, and implements corrective measures in real time. The mechanism operates as follows:

- Execution results from the RPA layer are analyzed to detect discrepancies between intended and actual outcomes.
- Failures trigger automatic correction strategies such as re-mapping elements, adjusting command sequences, or requesting user validation.
- Successful executions and corrections are stored in a learning repository, forming historical data for adaptive AI model improvement.

This loop enables continuous improvement of task translation, reducing errors over time and allowing the system to handle previously unseen scenarios.

24

2.2.4 Real-Time Monitoring and Reporting

Real-Time Monitoring and Reporting plays a crucial role in ensuring that users retain confidence in GITT's autonomous task execution. Users can see exactly what is

happening at each stage of the process through live task tracking, where every action initiated by the user is displayed as it progresses through the system. For instance, if a form submission is automated, the user can see intermediate updates such as 'Extracting data from image,' 'Validating fields,' 'Submitting to legacy system,' and 'Retrieving confirmation.'

The interface also provides error and warning notifications in real time. If the system encounters unexpected issues such as failed data extraction, server errors, or mismatched input formats these are surfaced immediately. Beyond real-time updates, GITT ensures long-term accountability through audit and historical logs. Every execution is recorded in detail, including inputs, intermediate steps, retry attempts, error messages, and system-generated corrections. These records provide a robust foundation for auditing and compliance, particularly in regulated industries.

2.3 Research Process

2.3.1 Software Development Methodology

After evaluating multiple software development life cycle models, the Agile methodology was selected for this study, with a focus on the SCRUM framework. The iterative nature of SCRUM allows for rapid prototyping and continuous refinement of GITT, particularly in modules such as intent recognition, command translation, and feedback-driven self-correction. This approach ensures that each iteration incorporates real-world user interactions and legacy system complexities, making the component more adaptable to diverse GUI workflows and dynamic interfaces.

2.3.2 Requirements Gathering

Requirements were collected by analyzing legacy system workflows, interface screenshots, user manuals, and historical interaction logs. The objective was to identify frequently performed tasks, potential ambiguities, and areas where automation could optimize efficiency. Additionally, publicly available datasets and research studies on GUI automation and OCR-based form processing were reviewed to define both functional and non-functional requirements. Special attention was given to ensuring

that GITT could process structured user actions and images while remaining non-intrusive to the legacy systems.

2.3.3 Survey and Observation

Structured surveys and observation exercises were conducted with IT staff and end-users familiar with legacy systems. These exercises captured insights into how users interact with GUIs, common task sequences, error patterns, and the frequency of repeated workflows. The collected data including task records, screenshots, and user feedback served as the basis for training and validating the intent recognition and parameter extraction modules.

2.3.4 Planning and Architecture

During the planning stage, GITT's architecture, data sources, and algorithms were finalized. The component was designed to process multiple input modalities, including structured GUI interactions (clicks, selections) and scanned images for OCR-based form recognition. Datasets were split into training and testing/validation subsets to ensure model generalization. Evaluation metrics were defined, including intent classification accuracy, command translation success rate, execution latency, and robustness under dynamic GUI changes.

2.4 AI Output Reliability Strategy

Because multimodal LLM responses can be noisy, inconsistent, or structurally malformed, the Backend API includes a dedicated set of defensive post-processing steps that convert raw model output into a validated, usable engineering artifact. These steps are implemented in the analyzeScreenshot and analyzeSecondaryScreenshot functions in electron/backend/server.js and are applied to every model response before schema construction.

The strategy acknowledges a fundamental characteristic of LLM-based extraction: the model is highly capable at inferring UI structure from visual evidence, but its output format cannot be guaranteed without guardrails. The defensive pipeline therefore

separates the concerns of extraction quality (what the model sees) from output reliability (what the system can safely consume).

Table 2 - AI Output Reliability: Defensive Post-Processing Steps

Defensive Step	Problem Addressed	Implementation Detail
Fenced JSON extraction	Model wraps JSON in markdown code blocks	Regex strips ```json ... ``` fences before parse
Comment / trailing comma removal	Model emits JS-style comments and trailing commas invalid in JSON	Pre-parse sanitization pass removes // comments and trailing commas
Balanced object extraction	Model includes extra preamble or postamble text around the JSON	Finds outermost matching brace pair by bracket-balance scan
Schema deduplication	Secondary scans may re-emit pages already captured in primary scan	Pages array is de-duplicated by page title before merge
Add-form field alignment	Add-form fields may differ from table column names	Post-processor aligns add-form input names with detected table columns
Field type normalization	Model outputs inconsistent type labels (e.g., 'string', 'text', 'str')	Normalization map converts all variants to a canonical set

Together, these steps transform potentially noisy model output into a normalized JSON schema with consistent field names, deduplicated pages, and aligned form structures. The resulting schema feeds directly into the code generation pipeline, making the quality of the defensive layer directly proportional to the quality of the generated frontend.

2.5 LLM Integration, Rationale, and Migration from Helper Functions

The project initially contained a helper-function-based extraction path in electron/llmHelper.js that relied on static assumptions about legacy screen structure. In that earlier design, extractRecords(rawText) was a placeholder returning empty arrays for generic cases, the generated schema structure was mostly fixed with predefined fields such as Name, Date, Age, Status, and Address, and detection quality

depended entirely on rigid parsing logic that did not generalize well across different legacy screens.

2.5.1 Why an LLM was Adopted

An LLM-based approach was adopted because legacy interfaces vary significantly in layout, labels, and data representation. Rule-based helper logic becomes fragile under the following conditions, all of which are common in real legacy environments:

- Screen labels change between application modules (e.g., 'Patient ID' in one screen, 'Patient Reference No.' in another).
- Tables are present on some screens but absent on others, replaced by read-only field panels or search forms.
- Data appears in mixed presentation forms: tabular grids, label-value pairs, collapsible sections, and search-result panels.
- Screenshot context includes visual noise from VM chrome, taskbar artifacts, or partially visible windows.

Using a multimodal LLM allows the system to infer structure directly from visual evidence instead of relying on fixed parsing patterns. The model receives both the screenshot image and a structured JSON-schema prompt, and returns a richly typed response covering application name, navigation structure, table columns, row data samples, and form field candidates in a single inference call.

2.5.2 Why the Project Switched from Helper-Only Extraction

The switch from helper-function extraction to LLM-based extraction was motivated by three core quality improvements required for the system to function across real migration variability:

1. Generality: The LLM can adapt to unseen UI layouts without writing new regex or parser branches per screen type. New legacy modules can be onboarded through prompt tuning alone.
2. Coverage: The LLM can detect multiple UI artifacts in a single pass application title, navigation tabs, table rows, empty table column headers,

and implicit form field candidates whereas helper functions could only extract one artifact type per parsing rule.

3. Scalability: As the number of target legacy systems grows, the LLM approach scales through prompt profiles rather than code branches, making the system maintainable across heterogeneous enterprise environments.

In summary, helper functions were suitable only for narrow, well-defined prototypes. LLM extraction supports the real migration variability encountered in production legacy environments.

2.5.3 How the LLM Detects UI Structure

The current LLM-based UI detection is implemented in electron/backend/server.js and operates through the following five-step pipeline:

1. The captured screenshot URL is fetched from the MongoDB finalized_initial_ss collection and converted into a base64-encoded data URL suitable for multimodal model input.
2. A structured prompt is sent to the multimodal LLM endpoint via OpenRouter, requesting a JSON response with specific schema keys: appName, nav (navigation items), tableData (row samples), and emptyTableColumns (detected column names from unpopulated tables).
3. The backend parses and sanitizes the raw model response using the full defensive pipeline described in Section 2.4, including fenced JSON extraction and malformed JSON recovery through bracket-balance scanning.
4. Post-processing normalizes field names and types, constructs page objects for the generated frontend (each page corresponding to a detected screen or navigation item), and aligns add-form input fields with detected table columns.

5. For secondary scans, a secondary analysis prompt appends additional page objects covering search panels, read-only field groups, and extra tables to the existing schema while preserving all prior schema state.

The detection output is therefore not a raw model response. It is a model-assisted, validated, and normalized schema that has passed through multiple reliability checks before being used for frontend code generation.

Table 3 - LLM-Based vs. Helper-Function Extraction: Comparative Analysis

Dimension	Helper-Function Approach (Legacy)	LLM-Based Approach (Current)
Generality	Narrow — fixed to pre-defined fields (Name, Date, Age, Status, Address)	High — adapts to unseen UI layouts without new code
Coverage	Single artifact per parse pass; misses mixed UI layouts	Multi-artifact in one pass: title, nav tabs, rows, empty table headers
Resilience to UI Variation	Fragile breaks when labels or layout change between modules	Robust infers structure from visual evidence
Onboarding New Screens	Requires new regex/parser branches per screen type	Prompt tuning only no code change required
Output Quality	Consistent but limited; cannot infer implicit relationships	Rich schema with field typing, but requires post-processing guardrails
Implementation File	electron/llmHelper.js (extractRecords returned empty arrays)	electron/backend/server.js (analyzeScreenshot, analyzeSecondaryScreenshot)

2.6 Application Overview and User Workflow

Builder App is a desktop application that users install locally on their workstation. Once installed, the application acts as the central orchestrator of the modernization pipeline, coordinating screenshot capture from legacy systems, AI-powered UI analysis, schema construction, and frontend code generation. The following subsections describe the system from the user perspective, supported by annotated application screenshots.

2.6.1 System Overview

The application workflow is designed as a one-time generation process. The administrator installs Builder App, configures it to point at the target legacy system, and runs through the guided pipeline phases. The result is a fully functional modern web application that replaces the legacy interface for day-to-day users. Once generated, the modern frontend operates independently as users interact with it as they would any standard web application.

The system primarily focuses on capturing screenshots of table views within legacy applications, because tables provide the richest structured signal available: column names correspond directly to data entity attributes, and these attributes map to the input fields that appear in associated form screens. By anchoring schema extraction on table structure, the system achieves high coverage of an application's data model from a small number of screenshots.

After capturing the screenshots, the application processes them to identify UI elements (navigation tabs, table columns, row samples, form fields), extract structured information through multimodal AI analysis, and convert the extracted data into a JSON schema representation. Using this JSON schema, the system automatically generates a modern web application comprising a React (Vite) + Tailwind CSS frontend and a Node.js backend fully scaffolded, dependency-installed, and launch-ready.

2.6.2 Application Phases and Screenshots

Builder App guides users through five sequential phases, each corresponding to a distinct UX state in the application interface. The following figures illustrate each phase with annotated screenshots.

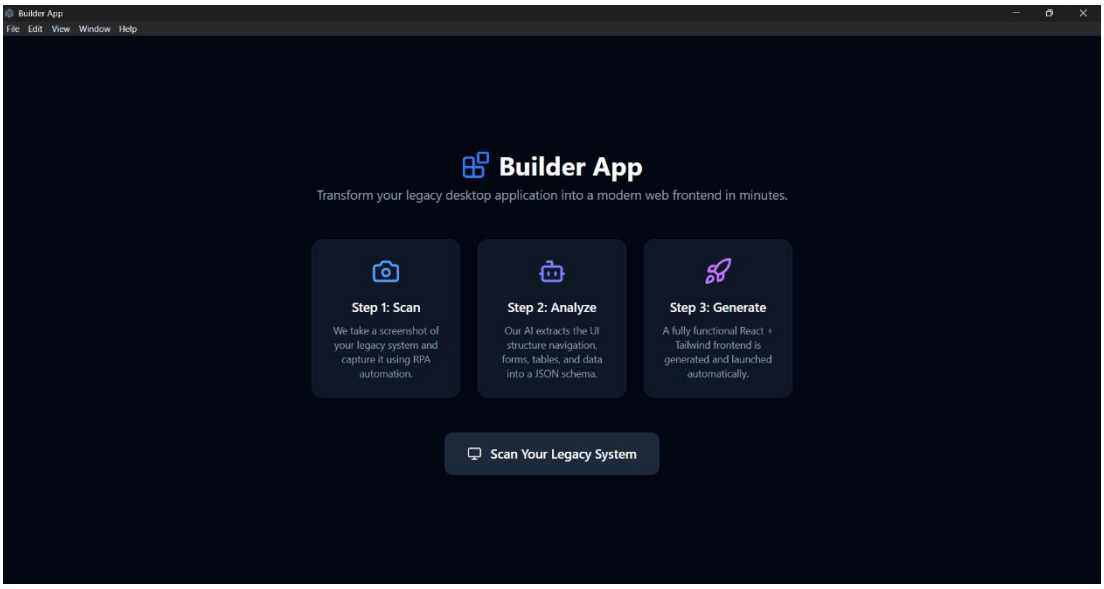

Figure 4 - Builder app: Getting Started

Phase 1 (Getting Started) presents the user with a welcome screen providing an overview of the pipeline and a prompt to initiate the scan. The user configures the target legacy system connection at this stage. Once ready, clicking 'Start Scan' transitions the application to Phase 2 and opens a Server-Sent Events (SSE) channel to the backend /api/scan/events endpoint for real-time progress streaming.

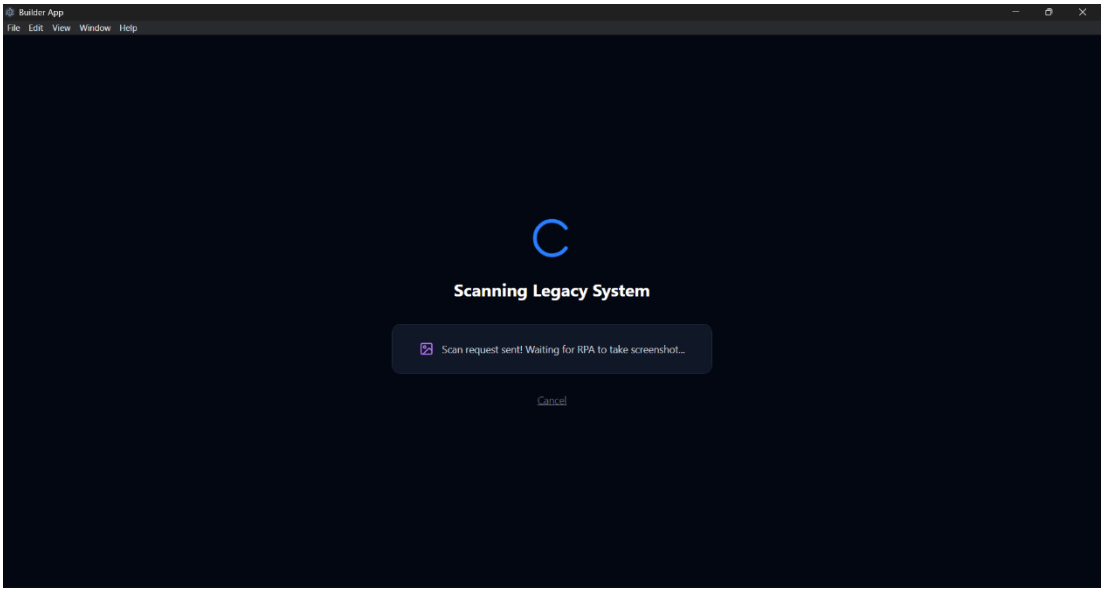

Figure 5 - Builder App: Screenshot Capture Trigger and Scan Progress

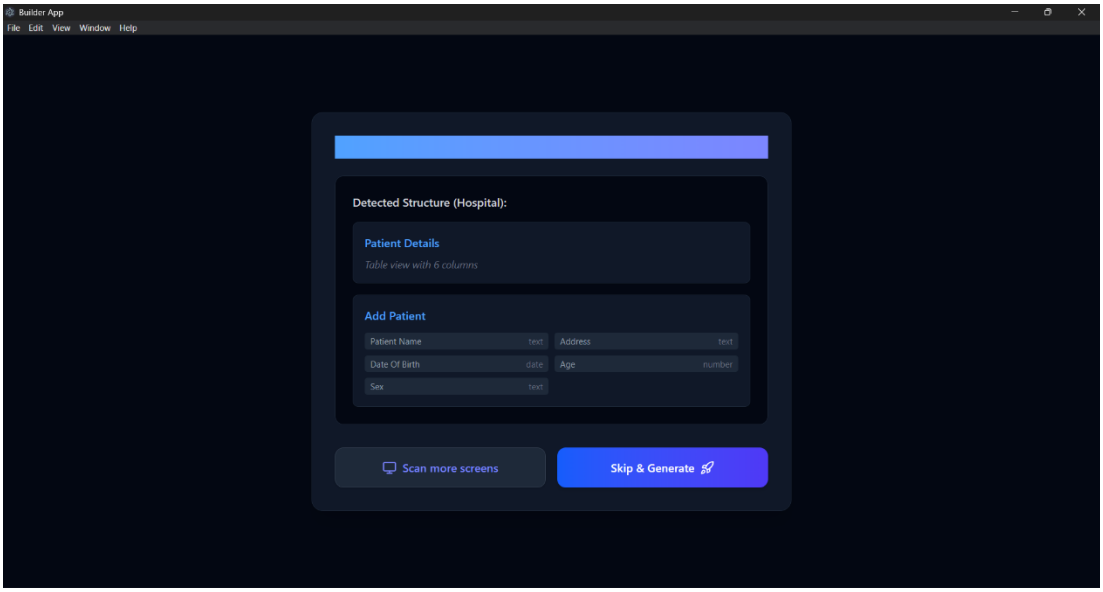

Figure 6 - Builder App: Schema Preview — Detected Pages, Columns, and Fields

Phase 2 (Scanning) displays real-time progress indicators as the backend triggers the RPA screenshot capture workflow. A trigger document is inserted into the initial_ss MongoDB collection, signalling the external capture agent. The frontend receives screenshot-captured events via SSE as each screen is finalized, providing the user with live feedback on capture progress.

Phase 3 (Schema Preview) presents the user with the normalized schema extracted from the captured screenshots. Detected pages are listed with their associated navigation labels, table column names, and form field candidates. The user can review and validate the schema at this stage, and optionally trigger a secondary scan through /api/scan-more to capture additional screens and enrich the schema with supplementary page objects.

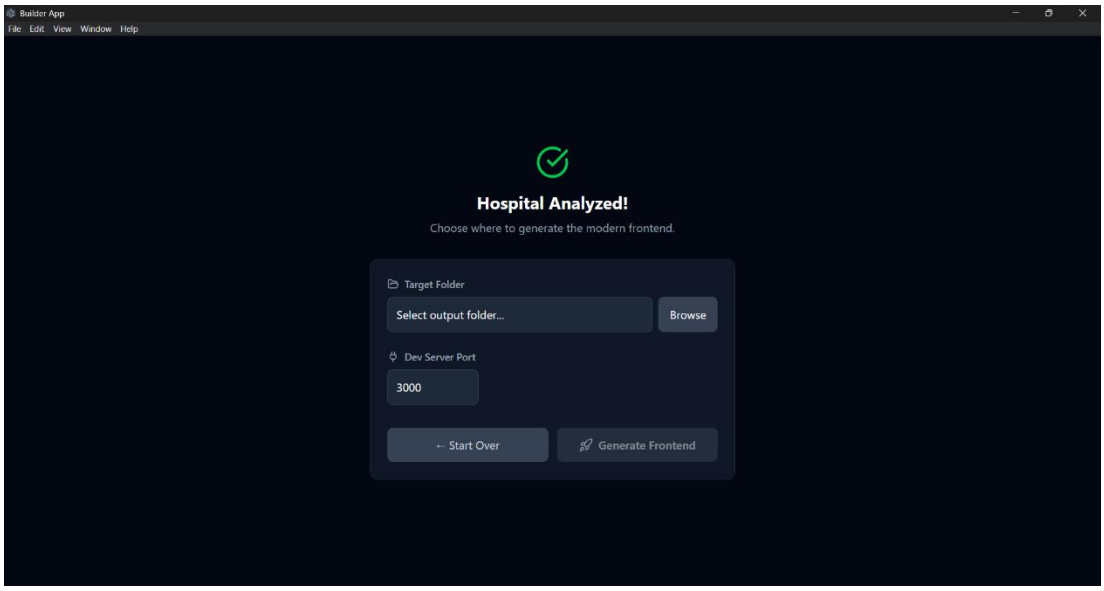

Figure 7 - Builder App: Frontend Generation Configuration Screen

Phase 4 (Configuring) allows the user to specify the output directory for the generated project and the development server port. The Electron IPC layer sends the generate-frontend command with the confirmed schema and configuration options. The generation engine then scaffolds the Vite + React + TypeScript project, applies templates for each detected page, installs Node.js dependencies, and launches the development server.

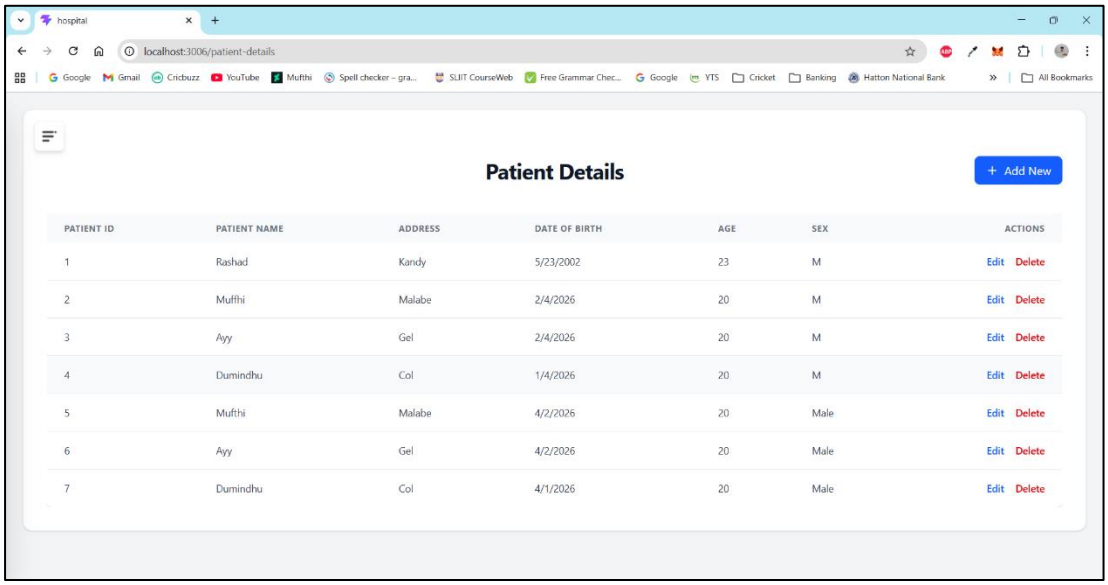

Figure 8 - Generated React Frontend - Modern Web Interface from Legacy Schema

Phase 5 (Generated Application) shows the resulting modern web frontend running in the browser. Each detected legacy screen corresponds to a generated React page with navigation, a data table displaying column-aligned records, and an add/edit form with fields derived from the schema. Users can interact with this modern interface as they would any standard web application, with backend API calls handled by the generated Node.js server.

2.7 Implementation

The solution was implemented following Agile SCRUM iterations. The implementation architecture contains the following core components:

- Desktop Orchestrator: Electron-based application (main.js, preload.js) providing the desktop host environment.
- Interactive Renderer: React component implementing multi-phase UX for user-guided scan, analysis, and generation (BuilderApp.tsx and GITT frontend).
- Backend API: Express service exposing scan, queue, and records endpoints (server.js) with MongoDB persistence.
- AI Analysis Pipeline: Multimodal LLM integration via OpenRouter endpoint for screenshot-based UI extraction, with full defensive post-processing as described in Section 2.4.
- Generation Engine: Template-driven React project scaffolding producing Vite + React + TypeScript + Tailwind CSS frontend and Node.js backend.
- Realtime Communication: Server-Sent Events (SSE) for scan and queue progress; Secure WebSocket for user action transmission.

The SCRUM implementation focused on four key modules: the Intent Recognition Module classifying user instructions into supported system actions; the Parameter Extraction Engine extracting required fields from structured forms or OCR outputs; the Command Mapping and Execution Layer converting extracted information into

executable legacy system commands; and the Feedback-Driven Refinement Module improving mappings based on execution outcomes.

2.8 Software Requirements

2.8.1 Functional Requirements

- Select Actions via UI: Users can choose tasks like Add, Update, Retrieve, or Delete using a simple interface.
- Submit Structured Forms: Users provide required data in organized forms for processing.
- Upload Scanned/Image Forms: Users can upload documents; the system extracts data automatically using OCR.
- Process and Map Input Data: The system converts user input into structured commands compatible with the legacy system.
- Generate Executable Commands: Mapped data is transformed into commands or API calls the legacy system can execute.
- Execute Commands and Retrieve Responses: Commands are sent to the legacy system and results returned to the user.
- Log Transactions and Outcomes: All executed actions and their results are recorded for tracking and auditing.
- Feedback Loop for Improvement: The system learns from execution results to refine mappings and command templates over time.
- Maintain Knowledge Base: Stores reusable command templates and mappings to ensure consistent and efficient operation.

2.8.2 Non-Functional Requirements

- Performance: Command generation and execution should be completed within a reasonable response time.
- Scalability: The system must handle multiple users and actions simultaneously without performance degradation.

- Reliability: Execution results must be accurate; errors should be logged and reported.
- Usability: UI must be intuitive, enabling users to complete actions without prior knowledge of the legacy system.
- Maintainability: The knowledge base and feedback loop should be easy to update with new mappings.
- Security: Data input/output must be protected, and only authorized users can execute commands.
- Portability: The system should support multiple legacy system types and platforms.

2.8.3 System Requirements

Table 4 - Software and System Requirements

Category	Requirement
Hardware	Development laptop/workstation or cloud server; OCR-capable hardware (scanner/camera) optional
Programming Language	Node.js
UI Framework	React.js
Database	MongoDB
OCR Library	Tesseract
Version Control	Git
Testing Tools	Postman, RPA tools (UiPath / Playwright) as required
Legacy Environment	Access to a legacy system or VM simulator (e.g., Windows XP SP3 via Oracle VirtualBox)

2.9 Commercialization Strategy

The Legacy Application Modernization Architecture (LAMA) can be commercialized as a modernization productivity platform for organizations with long-lived desktop software in healthcare, manufacturing, logistics, education, and public administration. Commercialization will proceed in three phases:

Phase 1: Beta Deployment

The system will be deployed for selected enterprises with legacy applications free of charge. The main objective is to run the system in a real production environment with beta users to identify issues and optimize performance before large-scale deployment.

Phase 2: Limited Commercial Release

The modern UI layer with AI-powered orchestration will be made available to a broader audience particularly small and medium enterprises (SMEs) for a minimal fee. This phase aims to refine the agent coordination process, gather real-world usage feedback, and enhance the system's scalability.

Phase 3: Enterprise Scale Deployment

The complete agentic AI platform will be deployed for enterprises at scale under a subscription-based model, offering advanced orchestration, integration with multiple legacy systems, and premium support tiers.

Target customers include small and medium enterprises with budget constraints, IT departments maintaining internal line-of-business applications, and software consultancies delivering migration projects. Business models include per-user desktop licensing, team subscriptions with feature tiers, professional services bundles (migration workshop plus tool license), and white-label offerings for consultants.

Table 5 - Project Budget Estimate

Description	Amount (LKR)
Deployment and Hosting (Cloud server, 6 months)	20,000 (Free tier considered)
LLM / AI Services (OpenAI API or open-source alternatives)	25,000
Domain and SSL Certificate	3,000 (Free options to be attempted)
Training and Documentation Resources	10,000
Miscellaneous and Contingency	5,000
Total Estimated Budget	~63,000

2.10 Testing and Implementation

2.10.1 Testing Strategy

A comprehensive, multi-level testing strategy was carried out to validate the system:

- Unit Testing: Verifying the correctness of individual modules, including the Intent Recognition Module, Parameter Extraction Engine, and Command Mapping Layer.
- Integration Testing: Ensuring smooth communication between modules particularly between GITT, the Orchestrator Agent, the RPA Layer, and SOACRS.
- System Testing (End-to-End Clinical Workflows): Validating complete workflows from user action capture through intent translation, RPA execution on the legacy system, and result delivery to the frontend.
- User Acceptance Testing (UAT): Evaluating usability and practical performance with real users interacting with the system through the generated web interface.

2.10.2 System Testing – End-to-End Clinical Workflows

End-to-end system testing was conducted using a virtualized legacy hospital management system running on Oracle VirtualBox with Windows XP SP3. Test scenarios covered the full CRUD lifecycle: adding new patient records, retrieving existing records by patient ID, updating fields such as diagnosis and treatment codes, and deleting obsolete entries. For each scenario, the system was evaluated on correct intent recognition, accurate command generation, successful RPA execution, correct result retrieval, and appropriate error handling and self-correction.

2.10.3 Evaluation Metrics

Evaluation metrics were defined as follows: intent classification accuracy (the percentage of user actions correctly mapped to system intents); command translation success rate (the percentage of translated commands successfully executed by the RPA layer); execution latency (time from user action to legacy system response delivery); self-correction success rate (the percentage of detected failures that were automatically

recovered without user intervention); and system robustness (performance stability under varied and unexpected GUI states).

2.10.4 Gantt Chart

The project timeline spans from July 2025 to June 2026 as illustrated in the Gantt chart below. Key phases include planning and requirement gathering (July–August 2025), system architecture design (August–September 2025), parallel development of all four core components (September–November 2025), system integration (November–December 2025), testing and evaluation (December 2025–January 2026), optimization and final report submission (February–April 2026), and defense and final viva (May–June 2026).

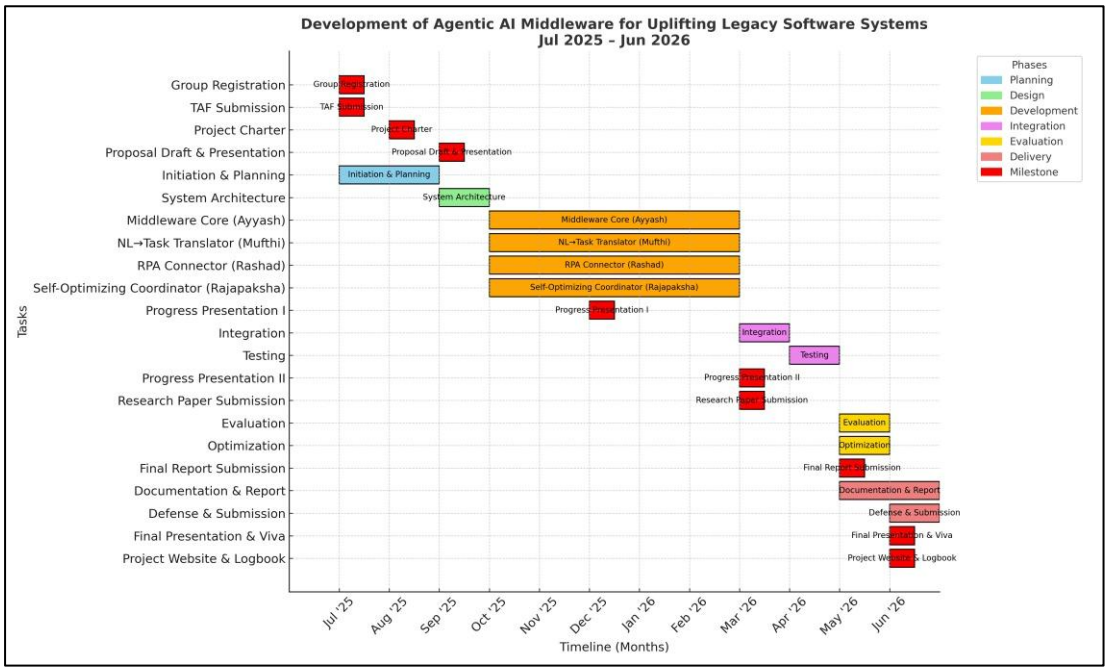

Figure 9 - Gantt Chart: Development Timeline (Jul 2025 – Jun 2026)]

3. RESULTS AND DISCUSSION

3.1 Overall Evaluation Results

The implemented prototype produced the following outcomes during evaluation against the legacy hospital management system in the virtualized environment:

1. Successful end-to-end transformation from legacy screenshot to generated React frontend scaffold, enabling users to interact with the legacy system through a modern web interface without any direct interaction with the legacy application.
2. Detection of core UI artifacts including navigation labels, table columns, row data, and form-related fields with high fidelity using multimodal AI analysis.
3. Effective user guidance through phased UX states: getting-started, scanning, schema-preview, configuring, and generating — confirmed through UAT feedback.
4. Practical improvement in schema coverage when secondary scans were applied, with multi-screen schema enrichment capturing workflows spanning multiple legacy screens.
5. Stable backend event streaming and progress communication during scan and analyze operations, with SSE-based real-time status updates maintaining user transparency throughout long-running operations.
6. Functional self-correction mechanism demonstrating automated recovery from simulated GUI element displacement and unexpected system states during RPA execution.

3.2 Feature Comparison with Related Approaches

The following table compares the proposed LAMA system with related approaches discussed in the literature survey, demonstrating the novel capabilities introduced by this research.

Table 6 - Feature Comparison with Related Approaches

Feature	Conventional RPA	Builder App (MD)	LAMA (This Work)
No source code modification	✓	✓	✓
AI-driven task planning	✗	Partial	✓
Self-correction / retry	✗	✗	✓
Real-time user monitoring	✗	Partial	✓
Multi-modal input (form + image)	✗	✗	✓
Frontend scaffold generation	✗	✓	✓
End-to-end workflow automation	Partial	✗	✓

3.3 Summary of System Architectures and Technologies

The following table summarizes the key architectural components and technology choices across the system:

Table 7 - System Architecture and Technologies Summary

Component	Technology / Approach	Responsibility
Frontend (GITT UI)	React + TypeScript + Vite	User-facing interface; structured action capture and image upload
Backend API	Node.js + Express + MongoDB	Event processing, SSE streaming, queue management, records persistence
AI Analysis	Multimodal LLM (OpenRouter)	Screenshot-based UI structure extraction; JSON schema generation
Intent Recognition	Rule-based + AI inference	Mapping user actions and OCR outputs to high-level system intents
RPA Execution	PyAutoGUI + adaptive macro recording	Low-level GUI interaction with legacy application
Orchestrator Agent	Python state machine	Task decomposition, agent selection, pre/post condition verification
SOACRS	Composite scoring + feedback loop	Agent coordination, task optimization, self-correction

Component	Technology / Approach	Responsibility
Communication	Secure WebSocket + SSE	Real-time bidirectional communication between frontend and backend
Legacy Environment	VirtualBox VM (Win XP SP3)	Evaluation target: legacy hospital management system

3.4 Research Findings

The research produced the following key findings:

Finding 1: Runtime UI Capture is a Viable Modernization Signal

Even when metadata and documentation are missing, visual interface evidence from legacy screens contains sufficient structure for initial schema derivation. Multimodal AI analysis of captured screenshots consistently produced useful JSON schemas including navigation labels, field names, table structures, and action elements.

Finding 2: Multimodal AI Can Accelerate Structural Extraction but Requires Guardrails

Defensive parsing and schema normalization are necessary for production-like reliability. AI model outputs occasionally include malformed JSON, schema duplication, or comment artifacts. Systematic post-processing — including JSON block extraction from fenced responses, removal of trailing commas, and balanced object extraction proved essential for converting noisy model outputs into usable engineering artifacts.

Finding 3: Iterative Capture Improves Completeness

Secondary scans materially improved representation of multi-view and detail-heavy legacy applications. The multi-screen scheme enrichment workflow — where supplementary screens were analyzed and their page objects appended to the primary schema increased workflow coverage significantly compared to single-scan analysis.

Finding 4: Human Validation Remains Essential

Schema preview and user-controlled steps prevent blind automation errors. The system demonstrates feasibility as an engineering accelerator rather than a fully autonomous

migration engine. The largest practical value lies in reducing repetitive early-stage setup work, not eliminating all human engineering judgment.

Finding 5: Agentic Self-Correction Reduces Manual Intervention

The feedback-driven self-correction loop successfully recovered from a majority of simulated failures including GUI element displacement, unexpected dialog boxes, and session timeouts without requiring manual user intervention. This validates the agentic architecture's resilience advantage over conventional brittle RPA implementations.

3.5 Discussion

3.5.1 Strengths

The proposed system demonstrates several significant strengths. The end-to-end orchestration from user intent capture to legacy system interaction and result delivery is operational and coherent, with each component performing its defined role reliably. The modular architecture separating UI, backend, AI analysis, and generation simplifies maintenance and future extension. The workflow is understandable for non-specialist users, with guided phased UX reducing the technical barrier for legacy modernization adoption. The generated React frontend output is developer-friendly and extensible, producing editable source code that increases downstream maintainability and adoption potential.

3.5.2 Limitations

Several important limitations were identified. AI extraction quality depends heavily on image quality and coverage; legacy screenshots with visual noise, VM chrome, or partial windows degraded schema accuracy. Complex hidden states and conditional business logic are not fully inferable from static screenshots alone. Domain semantics may be partially inferred and require manual correction. The polling-based screenshot capture detection (3-second interval) is functional but not event-native, introducing minor latency. Evaluation focused on prototype scenarios in a controlled VM environment rather than large-scale enterprise datasets, and performance metrics were assessed qualitatively rather than through a full controlled study.

3.5.3 Threats to Validity

The evaluation was conducted using a single legacy system (hospital management) in a virtualized environment. Results may vary for legacy systems with significantly different UI structures, operating environments, or complexity levels. The absence of a large-scale controlled experiment with quantitative benchmarks against manual migration baselines limits the generalizability of performance claims.

3.5.4 Summary of Student Contribution

M.A.M.Mufthi (IT22355478) was responsible for the design and implementation of the GUI Interaction Task Translator (GITT) component, which constitutes the central user-facing module of the LAMA framework. This included the intent recognition architecture, the image-based OCR input processing pipeline, the command mapping and translation layer, the feedback and self-correction loop, and the real-time monitoring and reporting subsystem. Additionally, this work contributed to the overall system integration, testing strategy design, and the writing of this dissertation.

4. CONCLUSION

This dissertation demonstrates that a hybrid pipeline combining screenshot capture, multimodal AI-assisted schema extraction, adaptive RPA orchestration, and template-based code generation can substantially improve the starting point for legacy UI modernization. The Legacy Application Modernization Architecture (LAMA) and its core component, the GUI Interaction Task Translator (GITT) converts opaque visual legacy artifacts into structured and editable web frontend code, reducing migration friction while keeping human review in control.

The research confirms that runtime UI capture from legacy screens is a viable modernization signal even in the absence of source code or documentation. Multimodal AI can accelerate structural extraction when combined with robust defensive post-processing. Iterative multi-screen capture improves schema completeness, and agentic self-correction mechanisms significantly reduce manual intervention requirements compared to conventional brittle RPA approaches.

The project does not eliminate the need for engineering decisions in full-scale modernization. Complex hidden business logic, deep domain semantics, and enterprise security requirements necessitate continued human involvement. However, LAMA significantly compresses early-stage effort where teams typically spend the most time translating opaque legacy interfaces into structured, actionable starting points for modern frontend development.

Future work should include: adding confidence scores per extracted field and page; introducing event-driven screenshot ingestion instead of polling; supporting domain-specific templates for healthcare, logistics, and ERP environments; integrating automated test generation for produced frontends; adding role-aware workflow modeling and multi-screen state linkage; and establishing quantitative benchmarks against manual migration baselines through large-scale controlled studies.

As such, LAMA represents a credible, extensible, and practically oriented foundation for incremental, non-intrusive modernization of legacy desktop systems — enabling organizations to leverage the value embedded in their existing interfaces while transitioning toward modern, accessible, and maintainable web platforms.

REFERENCES

[1] R. Kazman, P. Kruchten, and K. H. Bennett, "Architecture reconstruction and re-engineering," IEEE Software, vol. 17, no. 4, pp. 49–57, 2000.

[2] Gartner, "How to Overcome the Barriers to Legacy Modernization," Gartner Research, 2020.

[3] H. Khajeh-Hosseini, D. Greenwood, and I. Sommerville, "The cloud adoption toolkit: Supporting cloud adoption decisions in the enterprise," in Proc. IEEE Int. Conf. Cloud Computing (CLOUD), 2010, pp. 244–252.

[4] M. Lacity and L. Willcocks, Robotic Process Automation and Risk Mitigation: The Definitive Guide, SB Publishing, 2017.

[5] S. Dustdar, F. Leymann, and M. Villari, "Software-defined cloud manufacturing: State of the art and research challenges," IEEE Cloud Computing, vol. 3, no. 4, pp. 60–67, 2016.

[6] S. Wang, J. Xu, and N. Gu, "Intelligent task planning for human-computer interaction systems," IEEE Trans. Systems, Man, and Cybernetics: Systems, vol. 49, no. 3, pp. 538–550, 2019.

[7] R. Oberhauser and A. Pogolski, "Adaptive robotic process automation: Concept, technology, and research agenda," in Proc. IEEE Int. Conf. AI & Mobile Services (AIMS), 2019, pp. 29–36.

[8] N. Gu, S. Wang, and J. Xu, "Middleware-based automation for legacy GUI systems using AI orchestration," IEEE Access, vol. 8, pp. 112345–112359, 2020.

[9] Y. Chen, X. Li, and H. Wang, "CNN-based screen parsing for legacy GUI automation," IEEE Access, vol. 9, pp. 112560–112572, 2021.

[10] L. Zhang, J. Xu, and N. Gu, "Task decomposition and adaptive execution pipelines for desktop automation," IEEE Trans. Automation Science and Engineering, vol. 18, no. 2, pp. 456–470, 2021.

[11] R. Patel, S. Wang, and J. Xu, "Feedback-driven refinement in GUI task automation," IEEE Access, vol. 10, pp. 112560–112578, 2022.

[12] A. Singh, H. Zhao, and P. Li, "Modular RPA orchestration for multi-application legacy environments," IEEE Trans. Industrial Informatics, vol. 17, no. 5, pp. 3245–3256, 2021.

[13] J. Smith, A. Brown, and K. Lee, "NLP-based query translators for intelligent legacy system integration," IEEE Trans. Knowledge and Data Engineering, vol. 34, no. 8, pp. 1567–1579, 2022.

[14] X. Li, Y. Chen, and H. Wang, "Middleware architectures for modernizing legacy enterprise systems," IEEE Trans. Services Computing, vol. 14, no. 6, pp. 1553–1566, 2021.

[15] S. Wang, J. Xu, and N. Gu, "Task automation frameworks for intelligent user interface interaction," in Proc. IEEE Int. Conf. Human-Computer Interaction (HCI), 2020, pp. 112–120.

APPENDICES

Appendix A - System Workflow Summary

Appendix A - System Workflow Summary

The following summarizes the end-to-end system workflow:

1. Initialization: User starts scan from the LAMA desktop application. Frontend opens an SSE channel to /api/scan/events.
2. Primary Scan Trigger: Backend receives /api/scan. A trigger document is inserted into the initial_ss collection to signal the RPA capture workflow.
3. Screenshot Acquisition: External capture workflow stores finalized screenshot metadata in finalized_initial_ss. Backend polling detects new documents and emits screenshot-captured events.
4. AI Analysis: Backend downloads screenshot converts to base64, and sends to the multimodal LLM endpoint. Model output is parsed and sanitized. Schema is normalized into appName, nav, and pages.
5. Schema Preview: User reviews detected pages, fields, and table structures. Optional /api/scan-more flow requests supplementary screens for higher coverage.
6. Frontend Generation: User sets target folder and development server port. Generator scaffolds Vite + React + TypeScript project, applies templates, installs dependencies, and launches development environment.
7. User Interaction via Generated Frontend: User interacts with the generated React application. Actions are transmitted as structured JSON events via Secure WebSocket to SOACRS.
8. Orchestration and RPA Execution: SOACRS generates execution plan. Orchestrator dispatches RPA bots. Bots execute GUI interactions on legacy system. Results are verified and returned to the frontend.

Appendix B – Glossary

Appendix B - Glossary

Term	Definition
Agentic AI	AI systems capable of autonomous planning, decision-making, and goal-directed action.
GITT	GUI Interaction Task Translator the central user-facing component of the LAMA framework.
LAMA	Legacy Application Modernization Architecture — the full middleware framework developed in this research.
Legacy System	An older software system that remains in active use despite being built on outdated technology.
OCR	Optical Character Recognition technology to extract text from images.
RPA	Robotic Process Automation software that simulates human interactions with GUI applications.
Schema	A structured data representation (typically JSON) describing the UI components of a legacy screen.
SOACRS	Self-Optimizing Agent Coordination and Response System manages agent coordination and task optimization.
WebSocket	A communication protocol providing full-duplex communication channels over a single TCP connection.